



Automated Voltage Regulator System — [__main__](#)/ Program Reference

TABLE OF CONTENTS

 [File Overview](#)

 [__main__.ino](#)

 [Libraries Used](#)

 [Constants](#)

 [Enums](#)

 [Member Variables](#)

 [Methods](#)

[Constructor](#)

[Initialize\(\)](#)

[LCD Helpers](#)

[Servo Control](#)

[Homing](#)

[Current Sensor](#)

[Regulation Loop](#)

[Alarms & Recovery](#)

[Input Processing](#)

[Keypad Handler](#)

[runState\(\)](#)

[Serial Debug](#)

 File Overview

FILE	PURPOSE	LINES
<code>__main__.ino</code>	Entry point — hardware wiring, <code>setup()</code> , <code>loop()</code>	~41
<code>Machine.h</code>	Class definition — constants, enums, member declarations	~161
<code>Machine.cpp</code>	Class implementation — all logic and behavior	~739

 `__main__.ino` — Entry Point

This file is intentionally minimal. It handles only hardware instantiation and the Arduino `setup()` / `loop()` lifecycle.

What It Does

#	STEP	DETAIL
1	Includes	Pulls in <code>Machine.h</code> and all required libraries
2	Keypad Setup	Defines 4×4 key matrix, rows D6–D9, cols D10–D13
3	LCD Setup	I2C address <code>0x27</code> , 16 columns × 2 rows
4	Machine Object	Passes keypad and LCD by reference
5	<code>setup()</code>	Serial at 115200 baud → <code>machine.Initialize()</code>
6	<code>loop()</code>	<code>keypadPress()</code> → <code>runState()</code> every iteration

DESIGN NOTE

All logic lives inside the `Machine` class. This keeps `__main__.ino` as a thin wiring layer — no business logic, no state management.



Libraries Used

LIBRARY	INCLUDE	PURPOSE
Arduino Core	<code><Arduino.h></code>	Core functions (<code>millis()</code> , <code>analogRead()</code> , etc.)
Keypad	<code><Keypad.h></code>	4×4 matrix keypad scanning
Wire	<code><Wire.h></code>	I2C communication bus
LCD	<code><LiquidCrystal_I2C.h></code>	16×2 LCD via PCF8574 I2C expander
Servo	<code><Servo.h></code>	PWM servo control (continuous rotation)
EEPROM	<code><EEPROM.h></code>	Persistent storage for target and tolerance
Voltage Sensor	<code><ZMPT101B_DRIVER.h></code>	AC voltage sensing (non-blocking RMS, 4kHz)
Current Sensor	<code><ACS712-driver.h></code>	AC current sensing (blocking RMS, time-sliced)



Constants

Pin Assignments

CONSTANT	VALUE	DESCRIPTION
SERVO_PIN	3	PWM output to MG996R servo
BUZZER_PIN	5	Piezo buzzer for feedback & alarms
SERVO_POT_PIN	A2	Servo internal potentiometer (homing only)
VIN_SENSOR_PIN	A3	ZMPT101B #1 — input voltage
VOUT_SENSOR_PIN	A0	ZMPT101B #2 — output voltage
CSM_OUT_PIN	A1	ACS712 20A — output current

Physical Limits

CONSTANT	VALUE	DESCRIPTION
VARIAC_MAX_V	250	Maximum Variac dial position
POT_HOME_VALUE	5	ADC reading at 0V physical stop

EEPROM Addresses

CONSTANT	VALUE	DESCRIPTION
EEPROM_TOLERANCE_ADDR	0	Byte 0: tolerance (1–99, 0xFF = default)
EEPROM_TARGET_ADDR	1	Byte 1: target voltage (0–250, 0xFF = none)

Servo Pulse Timing

CONSTANT	VALUE	DESCRIPTION
PULSE_DURATION_MS	80	Spin time per pulse (ms)
SETTLE_DURATION_MS	150	Wait time after pulse for mechanics to settle
HOME_SPEED	1300	Servo μ s for homing (slow CCW)

Regulation

CONSTANT	VALUE	DESCRIPTION
DEFAULT_TOLERANCE_V	2	Default $\pm 2V$ if no EEPROM value
STALL_TIMEOUT	3000	Vout unchanged for 3s while pulsing → alarm
MIN_VIN_V	5	Below this = generator off, skip regulation
VOUT_AVG_SAMPLES	10	Moving average window size
CONFIRM_DURATION_MS	3000	Must stay outside tolerance this long before acting

Sensor Calibration

CONSTANT	VALUE	DESCRIPTION
ACS712_SENSITIVITY	0.100	V/A for ACS712-20A model
ZMPT_SENSITIVITY	500.0	ZMPT101B sensitivity factor
ZMPT_FREQUENCY	60.0	AC frequency in Hz



Enums

UIState — What the LCD shows

VALUE	DESCRIPTION
HOME	Main menu: [A] Configure / [B] View Status
CONFIGURE	Sub-menu: [A] Set Voltage / [B] Tolerance
INPUT_TARGET	Numeric input for target voltage (0–250)

INPUT_TOLERANCE	Numeric input for tolerance (1–99)
VIEW_STATUS	Sub-menu: [A] Input / [B] Output
VIEW_INPUT	Live display: Vin
VIEW_OUTPUT	Live display: Vout, Current, Power, Alarms

RegulatorState — Regulation mode

VALUE	DESCRIPTION
REG_IDLE	No target set — servo does nothing
REG_ACTIVE	Actively monitoring and correcting Vout
REG_UNDER_VOLTAGE	Variac maxed out — Vin too low to reach target
REG_OVER_VOLTAGE	Variac at minimum — Vin too high

ServoState — Pulse-and-wait FSM

VALUE	DESCRIPTION
SERVO_IDLE	Not moving — within tolerance
SERVO_PULSING	Spinning for <code>PULSE_DURATION_MS</code>
SERVO_SETTLING	Stopped — waiting <code>SETTLE_DURATION_MS</code> before re-reading



Member Variables

Hardware References

TYPE	NAME	DESCRIPTION
Keypad&	k	Reference to keypad object
LiquidCrystal_I2C&	lcd	Reference to LCD object
Servo	variacServo	Servo for PWM control
ZMPT101B	vinSensor	Voltage sensor on A3 (input)
ZMPT101B	voutSensor	Voltage sensor on A0 (output)
ACS712	currentSensor	Current sensor on A1

State Tracking

TYPE	NAME	DESCRIPTION
UIState	uiState	Current LCD screen
RegulatorState	regState	Current regulation mode
ServoState	servoState	Current pulse-and-wait phase

Servo Control

TYPE	NAME	DESCRIPTION
bool	servoAttached	Whether servo PWM is active
int	pulseDirection	+1 = CW (increase), -1 = CCW (decrease)
unsigned long	servoTimer	Timestamp for pulse/settle timing

Regulation

TYPE	NAME	DESCRIPTION
int	targetVoltage	User-set target (-1 = none)

int	toleranceV	Acceptable $\pm$ deviation from target
float	currentVin	Latest Vin reading
float	currentVout	Smoothed Vout (moving average)
float	rawVout	Raw Vout before averaging (for debug)
float	currentAmps	Latest current reading
float	currentWatts	Calculated power (Vout $\times$ Amps)

Stall Detection

TYPE	NAME	DESCRIPTION
float	lastVout	Previous Vout for change detection
unsigned long	lastVoutChangeTime	When Vout last changed by >1V

Voltage Smoothing

TYPE	NAME	DESCRIPTION
float[10]	voutSamples	Circular buffer for moving average
byte	voutSampleIdx	Current write position
float	smoothedVout	Computed average

Confirmation Window

TYPE	NAME	DESCRIPTION
unsigned long	outOfToleranceStart	When Vout first left tolerance (0 = inactive)
bool	isConfirmed	true = confirmation elapsed, servo can act

UI, Buzzer & LCD Throttle

TYPE	NAME	DESCRIPTION
String	<code>inputBuffer</code>	Digits typed by user
byte	<code>isLineFull</code>	LCD line counter for auto-clear
bool	<code>backlightOn</code>	LCD backlight state
unsigned long	<code>buzzerOnMillis</code>	When buzzer was turned on
bool	<code>buzzerActive</code>	Whether buzzer is sounding
unsigned long	<code>lastLCDTick</code>	Last LCD/debug update (500ms throttle)



Method Summary

PUBLIC Methods

METHOD	DESCRIPTION
<code>Machine(Keypad&, LiquidCrystal_I2C&)</code>	Constructor — initializes all members
<code>void Initialize()</code>	Hardware setup, EEPROM load, sensor calibration, homing
<code>void runState()</code>	Main tick — sensors, regulation, UI (every <code>loop()</code>)
<code>void keypadPress()</code>	Read keypad, route key to handler

PRIVATE Methods

METHOD	DESCRIPTION
<code>displayLCD(bool, byte, byte, String)</code>	Write text with optional clear

<code>displayLCD(byte, byte, String)</code>	Write text with auto-clear
<code>processTargetInput(char)</code>	Handle input for target voltage
<code>processToleranceInput(char)</code>	Handle input for tolerance
<code>servoStart(int)</code>	Attach and write microseconds
<code>servoStop()</code>	Stop → delay 5ms → detach
<code>homeToZero()</code>	Spin CCW until POT ≤ home value (15s timeout)
<code>regulateLoop()</code>	Vout closed-loop: confirm → pulse → settle
<code>checkAlarms()</code>	Detect stall → trigger alarm
<code>checkAlarmRecovery()</code>	Auto-recover from alarm states
<code>readCurrent()</code>	ACS712 AC RMS read (~20ms blocking)
<code>printDebug()</code>	10-column fixed-width Serial output

Constructor LINE 6-45

Initializes all member variables to safe defaults. Sensor objects use the member initializer list:

```
vinSensor(VIN_SENSOR_PIN, ZMPT_FREQUENCY)    // A3, 60Hz
voutSensor(VOUT_SENSOR_PIN, ZMPT_FREQUENCY)   // A0, 60Hz
currentSensor(CSM_OUT_PIN, 5.0, 1023)         // A1, 5V ref, 10-bit
```

Key defaults: `targetVoltage = -1` (no target), `toleranceV = 2`, all sensor values = `0`.

Initialize() LINE 50-105

Boot sequence executed once in `setup()` :

Power ON

- └ Pin modes (buzzer = OUTPUT, POT = INPUT)
- └ LCD init + backlight + "System starting!"
- └ ZMPT101B sensitivity set (both sensors)
- └ ACS712 sensitivity + `calibrate()` (auto-zero, no load)
- └ Load tolerance from EEPROM byte 0
- └ Load target from EEPROM byte 1
 - └ If valid (0-250) → set `REG_ACTIVE` automatically
- └ `homeToZero()` → servo spins CCW until POT ≤ 5
- └ 1 second delay
- └ Show HOME screen

⚠ IMPORTANT

`currentSensor.calibrate()` must run with **no load** connected to get an accurate zero-point reading.

LCD Helpers LINE 110-128

Version 1: `displayLCD(bool clear, byte col, byte line, String text)`

If `clear = true` → clears entire LCD, resets line counter, then prints text.

Version 2: `displayLCD(byte col, byte line, String text)`

Auto-clears when both lines written (`isLineFull ≥ 2`). Prevents stale text.

Servo Control LINE 133-148

servoStart(int speed)

Attaches servo to `SERVO_PIN` if not attached, then writes microseconds.

VALUE	ACTION
1500	Stop
> 1500	Clockwise (increase voltage)
< 1500	Counter-clockwise (decrease voltage)

servoStop()

Writes `1500` (stop), waits 5ms, then **detaches** to prevent MG996R drift from unbalanced internal resistors.

homeToZero() LINE 153-196

Moves servo to the physical 0V position on boot using POT feedback.

```
Read POT (A2)
├ POT ≤ 5 → "Already at 0V" → skip
└ POT > 5 → LCD: "Homing to 0V..."
    ├── Spin CCW at 1300μs (slow)
    ├── Monitor POT continuously
    ├── POT ≤ 5 → STOP → "Homed to 0V" ✓
    └ 15s timeout → STOP → "Home failed!" ⚠
```

NOTE

This is the **only blocking function** besides `readCurrent()` . Runs once during boot.

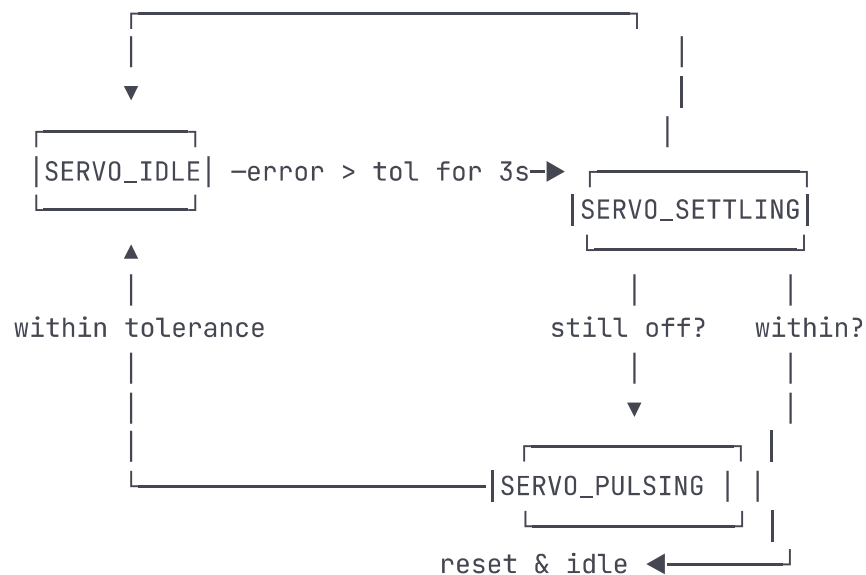
readCurrent() LINE 201-203

Calls `currentSensor.readCurrentAC(60)` from the ACS712-driver library.

PROPERTY	VALUE
Block time	~20ms (one 60Hz AC cycle)
Call interval	Every 500ms (time-sliced in <code>runState</code>)
Returns	Current in Amperes (RMS)

regulateLoop() LINE 209-286

The core regulation engine. Uses a **3-state FSM** (pulse-and-wait):



Proportional Speed

ERROR	SPEED	SERVO MS
> 30V	200 (fast)	1700 or 1300
10–30V	100 (medium)	1600 or 1400
< 10V	50 (slow)	1550 or 1450

Confirmation Window

Vout must stay **continuously outside tolerance for 3 seconds** before the servo acts. If Vout returns to tolerance at any point, the timer resets. This prevents hunting from noise spikes.

Alarms & Recovery LINE 288–338

checkAlarms()

Detects **stalled** conditions — servo pulsing but Vout not changing.

CONDITION	ALARM	MEANING
<code>pulseDirection = +1</code> & stall	<code>REG_UNDER_VOLTAGE</code>	Variac maxed out, Vin too low
<code>pulseDirection = -1</code> & stall	<code>REG_OVER_VOLTAGE</code>	Variac at minimum, Vin too high

Stall = Vout unchanged for `STALL_TIMEOUT` (3s) while servo is active.

checkAlarmRecovery()

Under Voltage: `maxAchievable = Vin × (250/220)` . If $\geq$ target → recovered.

Over Voltage: If `currentVout ≤ target + tolerance` → recovered.

Recovery is **automatic** — no user intervention needed.

Input Processing LINE 344-441

processTargetInput(char key)

KEY	ACTION
0-9	Append digit (max 3 digits)
*	Backspace — remove last digit
A	Confirm: validate 0-250, save EEPROM, start regulation
D	Cancel: clear buffer, back to CONFIGURE

processToleranceInput(char key)

Same pattern: max 2 digits, valid range 1-99, saves to EEPROM, returns to CONFIGURE.

keypadPress() LINE 446-543

Routes keypad input based on current `uiState` . `#` toggles backlight globally.

Navigation Tree

```
HOME
├─ A → CONFIGURE
│   │   ┌─ A → INPUT_TARGET
│   │   │   ┌─ A → Confirm → VIEW_STATUS
│   │   │   └─ D → Cancel → CONFIGURE
│   │   └─ B → INPUT_TOLERANCE
│   │       ┌─ A → Confirm → CONFIGURE
│   │       └─ D → Cancel → CONFIGURE
│   └─ D → HOME
└─ B → VIEW_STATUS
    ┌─ A → VIEW_INPUT
    └─ D → VIEW_STATUS
```

```
└─ B → VIEW_OUTPUT
└─   └─ D → VIEW_STATUS
└─ D → HOME
```

Every keypress triggers a **100ms buzzer click** for tactile feedback.

runState() LINE 548-649

Main tick function called every `loop()` . Divided into 4 sections:

Section 1: Hardware Ticks

Buzzer auto-off after 100ms.

Section 2: Sensor Ticks (Non-blocking)

SENSOR	METHOD	NOTES
Vin	<code>vinSensor.update()</code>	Non-blocking, with ADC MUX priming
Vout	<code>voutSensor.update()</code>	Non-blocking → 10-sample moving average
Current	<code>readCurrent()</code>	Every 500ms, blocks ~20ms

Section 3: Regulation Ticks

`REG_ACTIVE` → `regulateLoop()` + `checkAlarms()`

Alarm states → `checkAlarmRecovery()`

Section 4: UI Ticks (every 500ms)

Serial debug via `printDebug()` , LCD updates for `VIEW_INPUT` and `VIEW_OUTPUT` only.

printDebug()

LINE 654-738

Fixed-width columnar debug output at 115200 baud, every 500ms:

```
Vin: 220V | Vout:  24V | V_avg:  25V | I: 300mA | POT: 342 | Tgt:  25V | Tol: +-
◀────────────────────────────────────────────────────────────────────────────────▶
```

COLUMN	SOURCE	DESCRIPTION
Vin	currentVin	Generator input voltage
Vout	rawVout	Raw output voltage (pre-average)
V_avg	currentVout	Smoothed output (regulation uses this)
I	currentAmps	Auto-switches A/mA below 1A
POT	analogRead(A2)	Servo pot raw ADC
Tgt	targetVoltage	User target or ---
Tol	toleranceV	± tolerance setting
Reg	regState	IDLE / ACTIVE / UNDER_V / OVER_V
Srv	servoState	IDLE / PULSE / SETTLE
Cfm	outOfToleranceStart	Timer progress or ---

IMPLEMENTATION

Uses `sprintf()` with `%4d` for right-justified numbers. All string literals wrapped in `F()` macro to save SRAM.



Execution Flow

Every loop() Iteration (~1ms)

```
loop()
├─ keypadPress()           ← scan keypad, route to handler
└─ runState()
    ├─ [1] Buzzer auto-off check
    ├─ [2] Sensor reads
    │   ├─ Vin (non-blocking, ADC MUX primed)
    │   ├─ Vout (non-blocking + 10-sample moving average)
    │   └─ Current (every 500ms, blocks 20ms)
    ├─ [3] Regulation
    │   ├─ REG_ACTIVE → regulateLoop() + checkAlarms()
    │   └─ ALARM → checkAlarmRecovery()
    └─ [4] UI (every 500ms)
        ├─ printDebug() → Serial (115200 baud)
        └─ LCD update (VIEW_INPUT or VIEW_OUTPUT only)
```

Timing Budget

TASK	FREQUENCY	DURATION
Keypad scan	Every loop	~50µs
ZMPT101B update (×2)	Every 250µs	~100µs
ACS712 read	Every 500ms	~20ms (blocking)
Regulation FSM	Every loop	~10µs
Serial debug	Every 500ms	~1ms
LCD update	Every 500ms	~2ms

